DATA DIGGER

E-Commerce Relational Database Architecture & Query Implementation

Student Name	Priyal Patel
Course & Subject	SQL & Relational Database Management System (RDBMS)
DBMS Platform	MySQL Server & phpMyAdmin (Apache / XAMPP)
GitHub Repository	github.com/priyal-24/Data-Digger

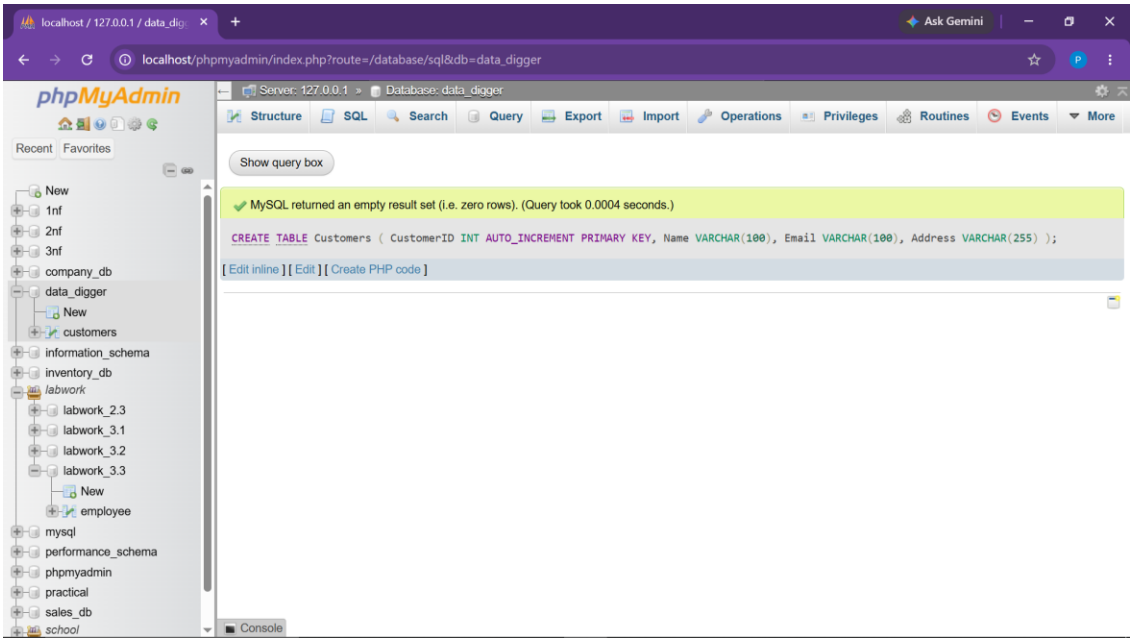
Executive Summary & Overview

Data Digger is a comprehensive relational database system designed to simulate core e-commerce backend operations. The schema integrates four relational entities (Customers, Orders, Products, and OrderDetails) enforcing primary and foreign key constraints. This practical report documents full end-to-end execution of DDL schema definitions, CRUD operations, date-interval queries, aggregate metrics, and multi-table relational JOINs.

1. DATABASE SCHEMA & TABLE CREATION

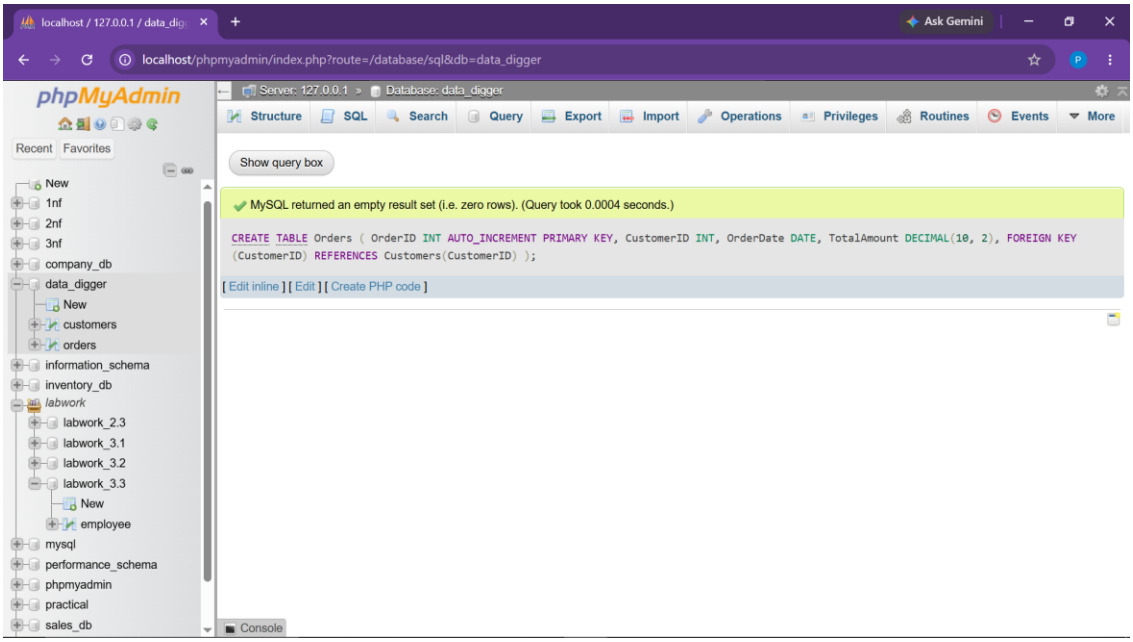
- **1.1 Create Customers Table**

```
CREATE TABLE Customers (
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100),
    Address VARCHAR(255)
);
```



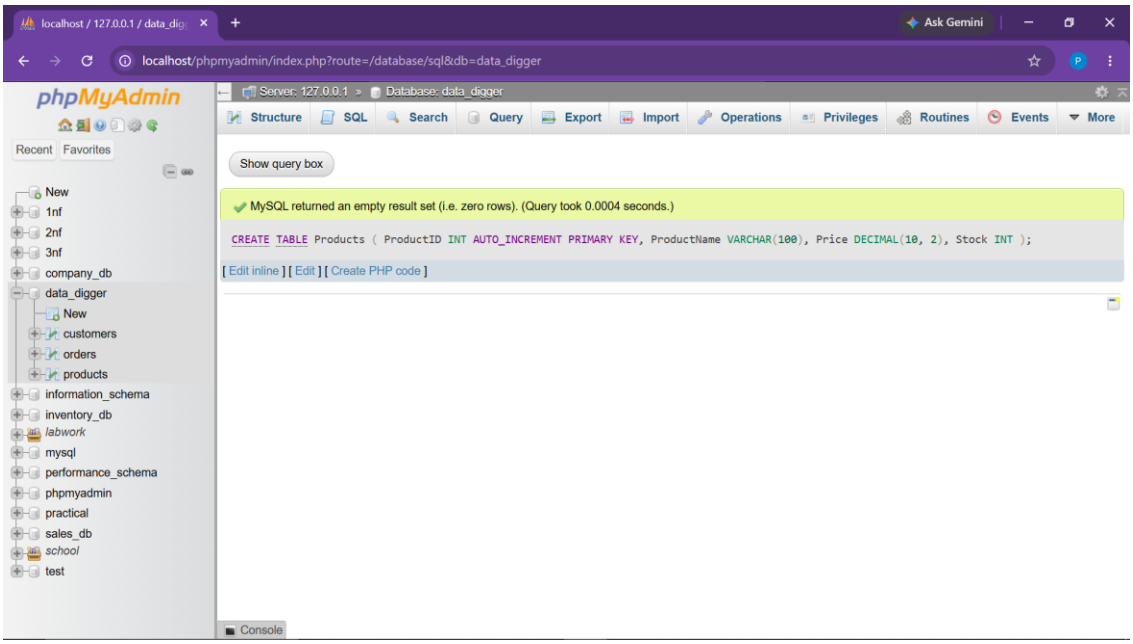
- **1.2 Create Orders Table**

```
CREATE TABLE Orders (
    OrderID INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    TotalAmount DECIMAL(10, 2),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```



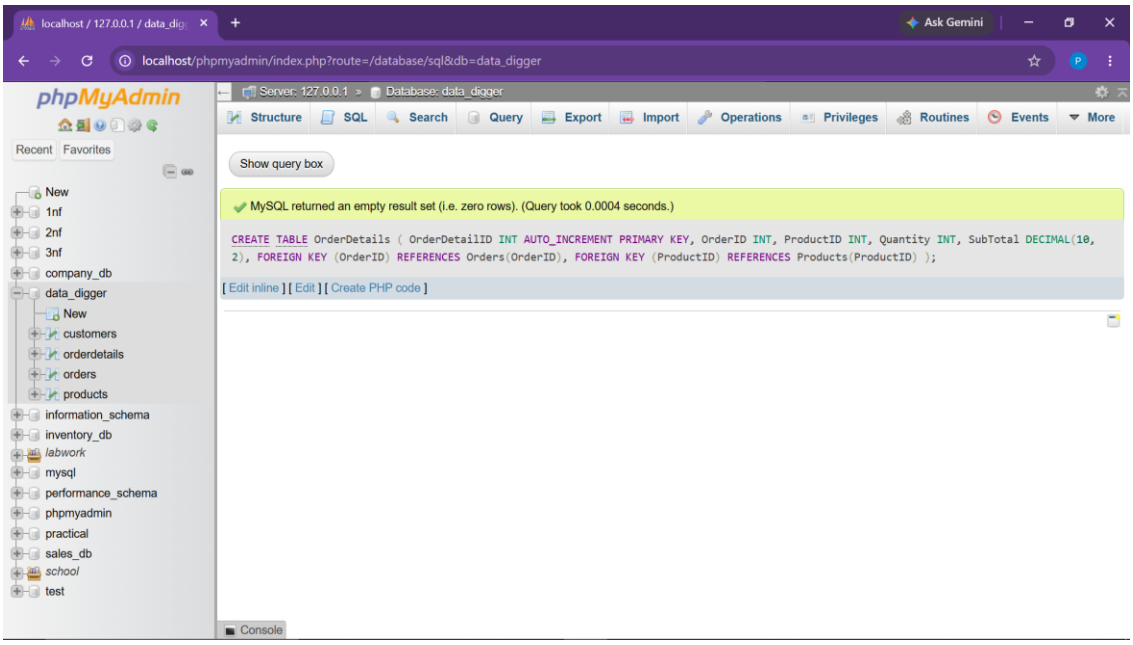
- ## 1.3 Create Products Table

```
CREATE TABLE Products (
    ProductID INT AUTO_INCREMENT PRIMARY KEY,
    ProductName VARCHAR(100),
    Price DECIMAL(10, 2),
    Stock INT
);
```



- **1.4 Create OrderDetails Table**

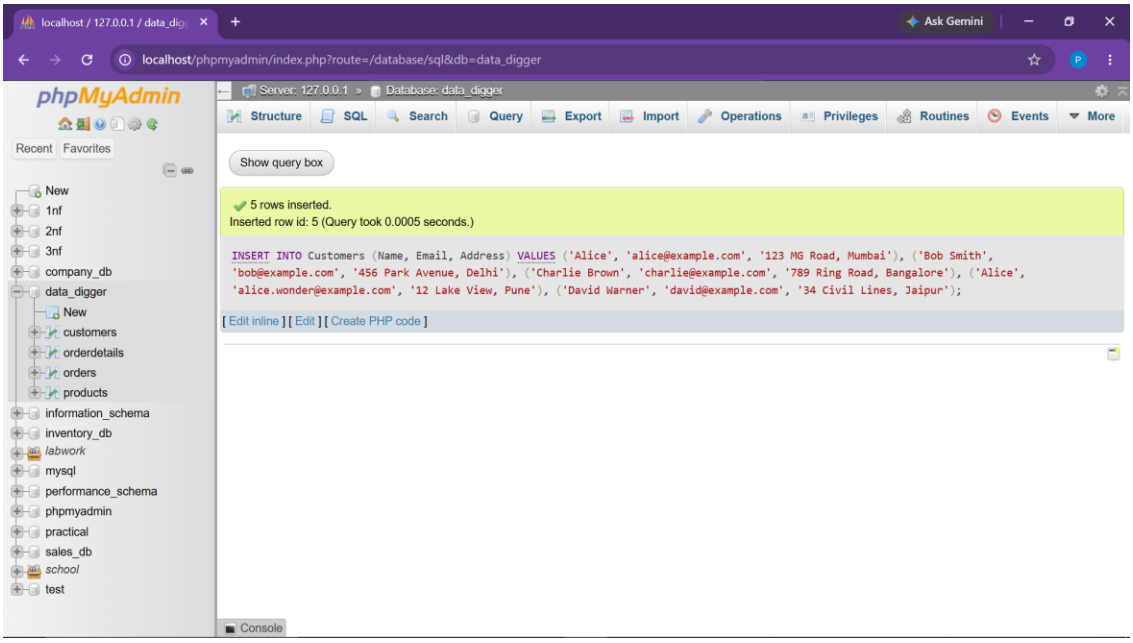
```
CREATE TABLE OrderDetails (
    OrderDetailID INT AUTO_INCREMENT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT,
    SubTotal DECIMAL(10, 2),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```



2. CUSTOMERS TABLE - DATA MANIPULATION & QUERIES

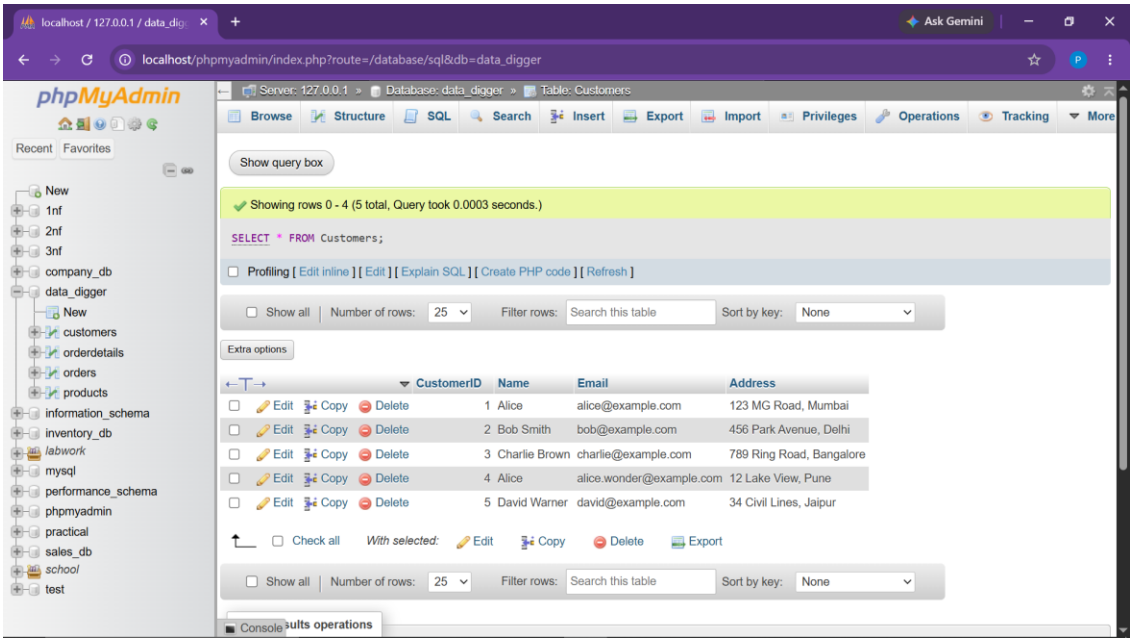
- **2.1 Insert Records into Customers**

```
INSERT INTO Customers (Name, Email, Address) VALUES
('Alice', 'alice@example.com', '123 MG Road, Mumbai'),
('Bob Smith', 'bob@example.com', '456 Park Avenue, Delhi'),
('Charlie Brown', 'charlie@example.com', '789 Ring Road, Bangalore'),
('Alice', 'alice.wonder@example.com', '12 Lake View, Pune'),
('David Warner', 'david@example.com', '34 Civil Lines, Jaipur');
```



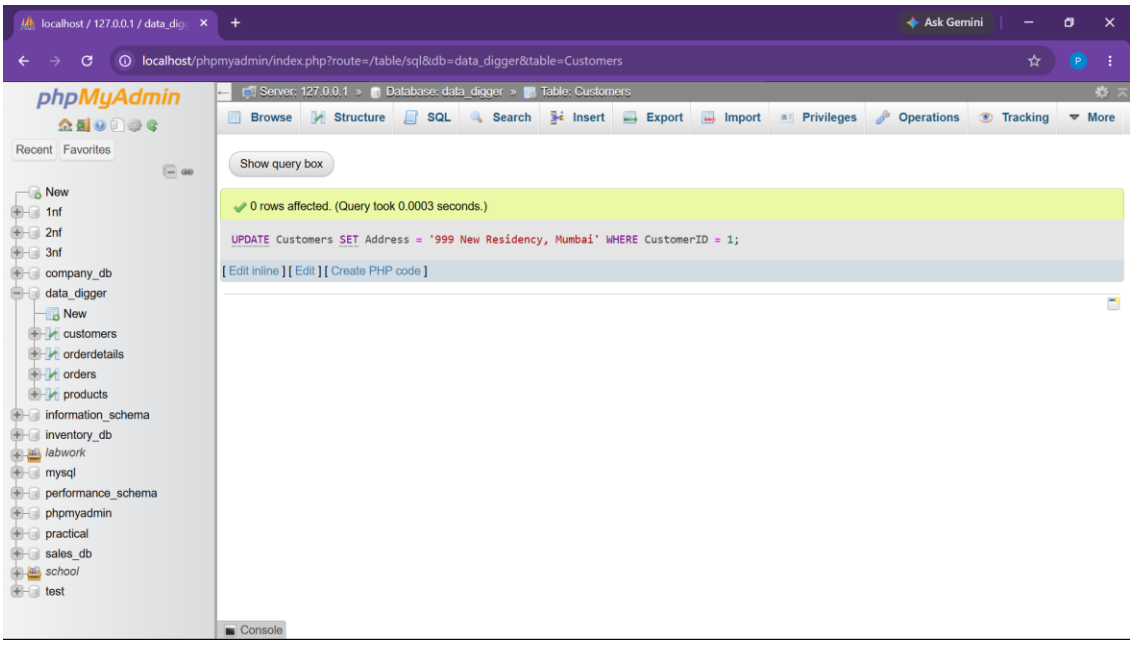
2.2 Retrieve All Customers

```
SELECT * FROM Customers;
```



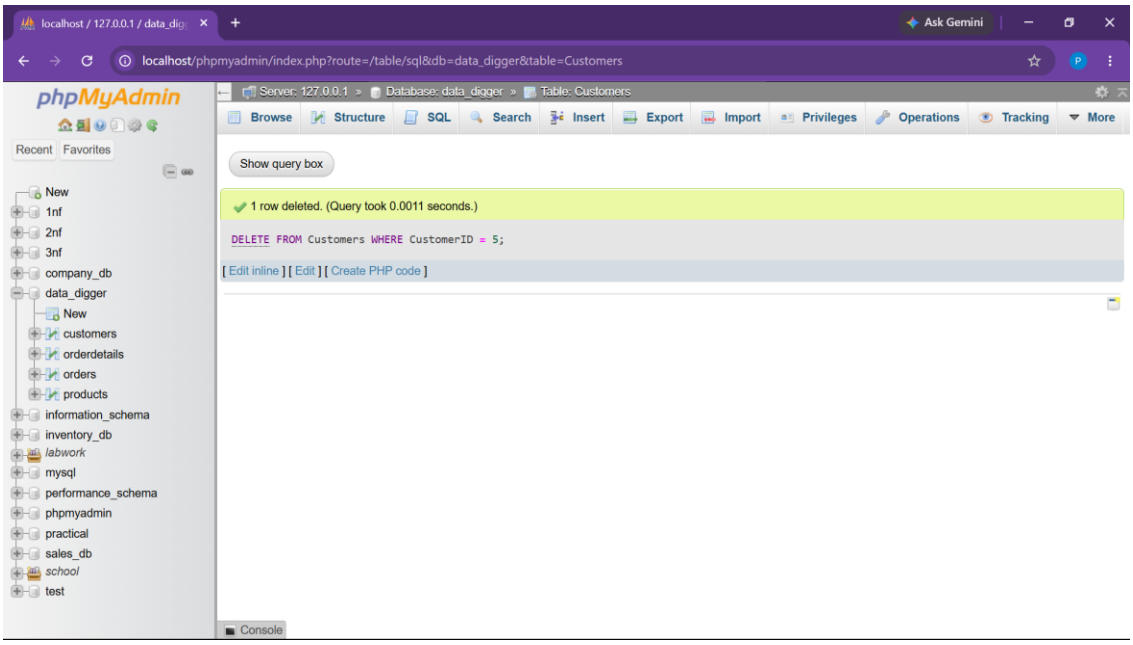
- **2.3 Update Customer Address (CustomerID = 1)**

```
UPDATE Customers
SET Address = '999 New Residency, Mumbai'
WHERE CustomerID = 1;
```



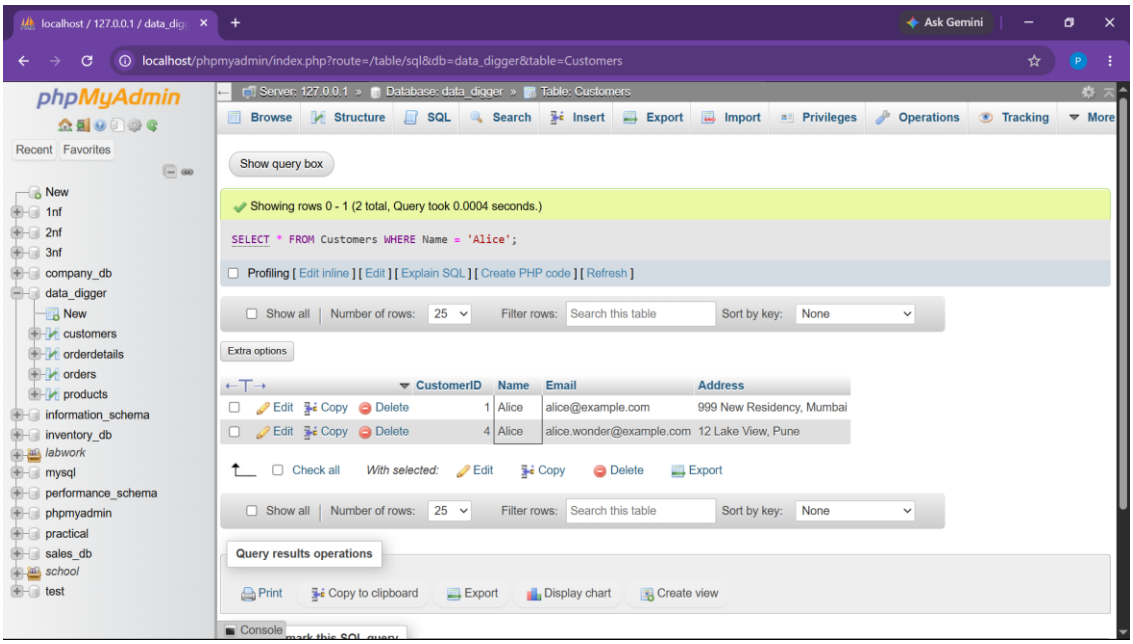
- **2.4 Delete a Customer using CustomerID (CustomerID = 5)**

```
DELETE FROM Customers
WHERE CustomerID = 5;
```



- **2.5 Retrieve Customers by Name ('Alice')**

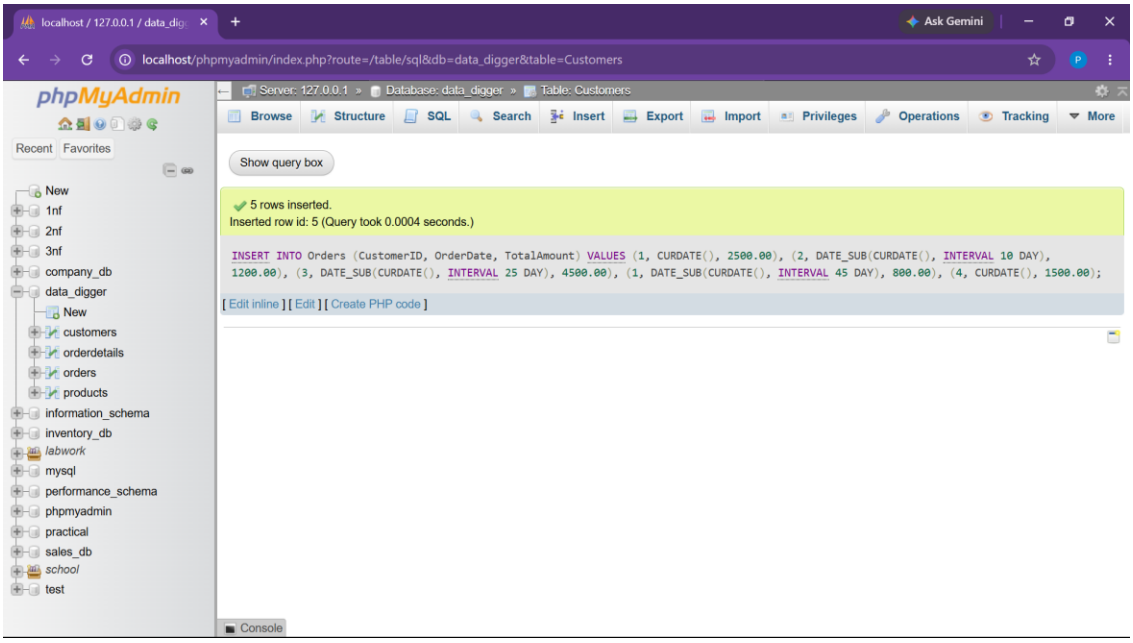
```
SELECT * FROM Customers
WHERE Name = 'Alice';
```



3. ORDERS TABLE - DATA MANIPULATION & QUERIES

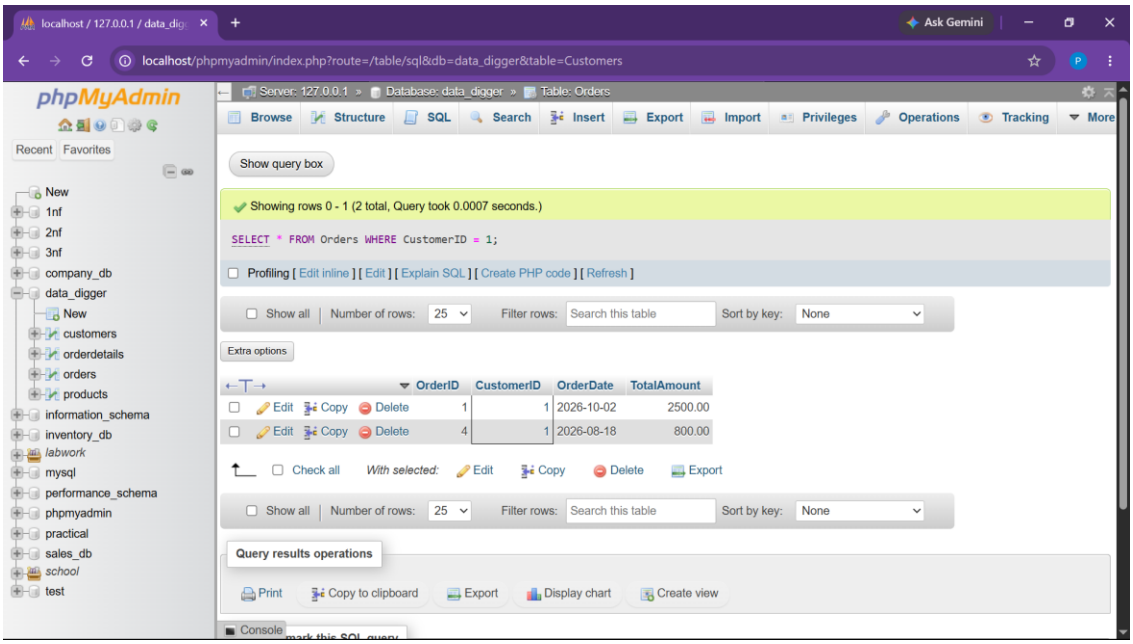
- **3.1 Insert Sample Records into Orders**

```
INSERT INTO Orders (CustomerID, OrderDate, TotalAmount) VALUES
(1, CURDATE(), 2500.00),
(2, DATE_SUB(CURDATE(), INTERVAL 10 DAY), 1200.00),
(3, DATE_SUB(CURDATE(), INTERVAL 25 DAY), 4500.00),
(1, DATE_SUB(CURDATE(), INTERVAL 45 DAY), 800.00),
(4, CURDATE(), 1500.00);
```



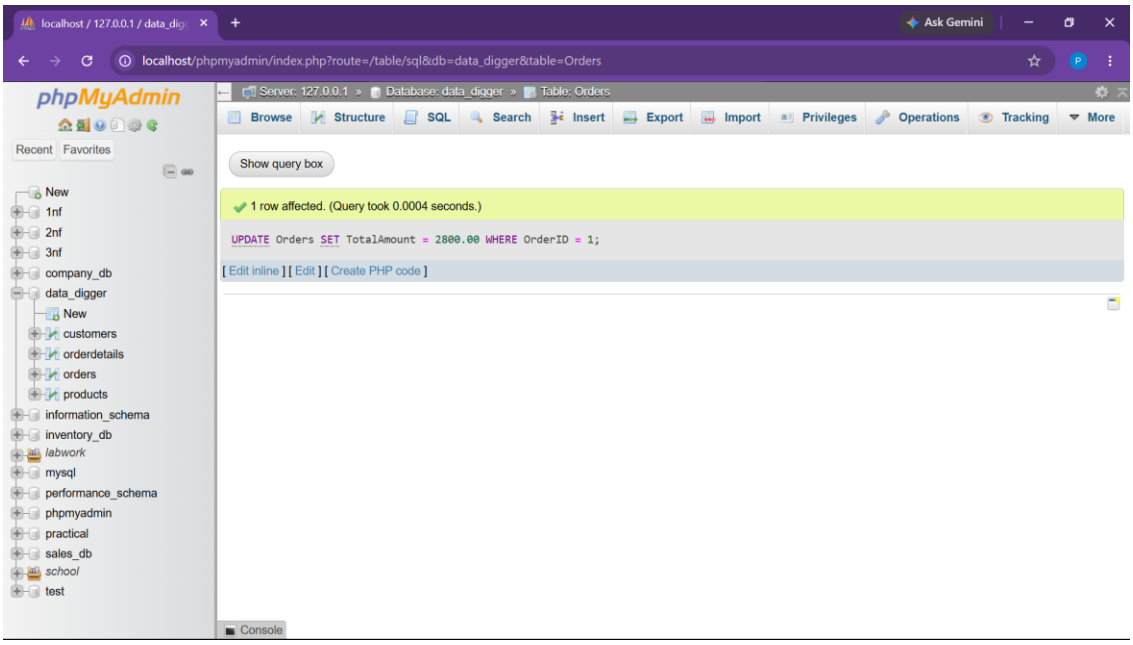
- ### 3.2 Retrieve Orders for a Specific Customer (CustomerID = 1)

```
SELECT * FROM Orders
WHERE CustomerID = 1;
```



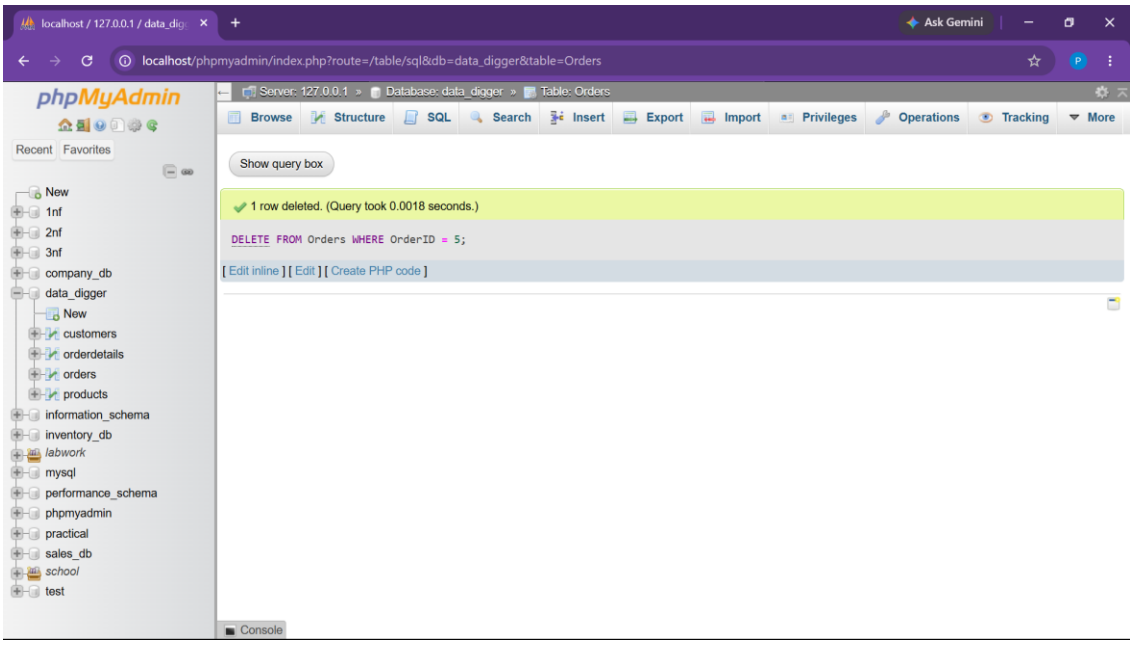
- **3.3 Update TotalAmount for Specific Order (OrderID = 1)**

```
UPDATE Orders
SET TotalAmount = 2800.00
WHERE OrderID = 1;
```



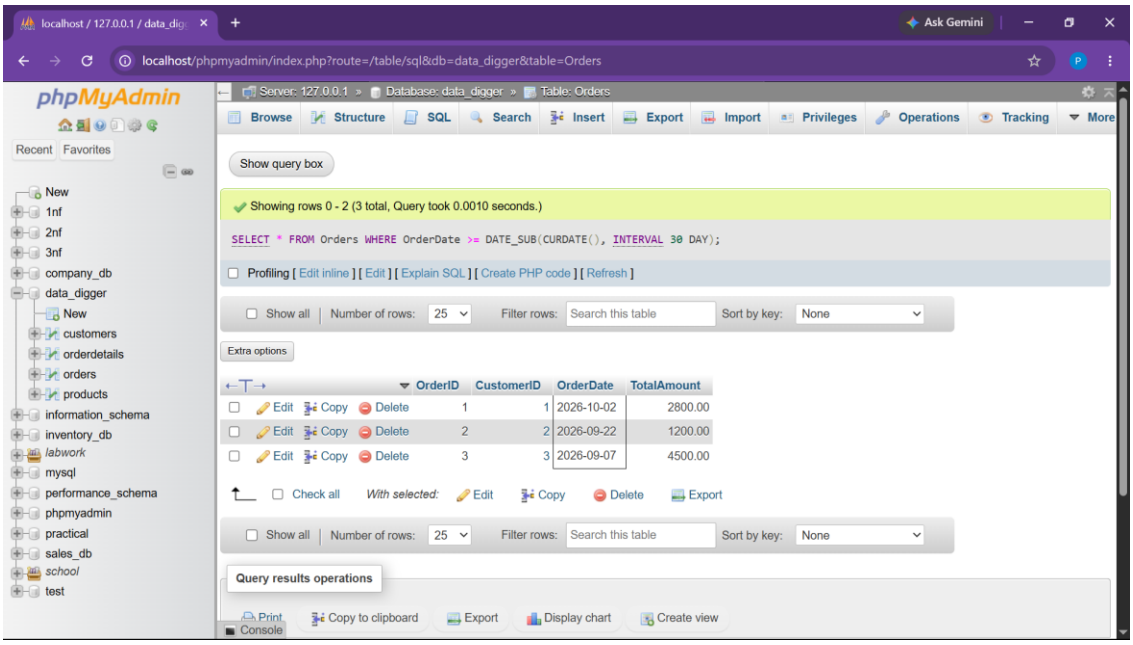
- **3.4 Delete an Order using its OrderID (OrderID = 5)**

```
DELETE FROM Orders
WHERE OrderID = 5;
```



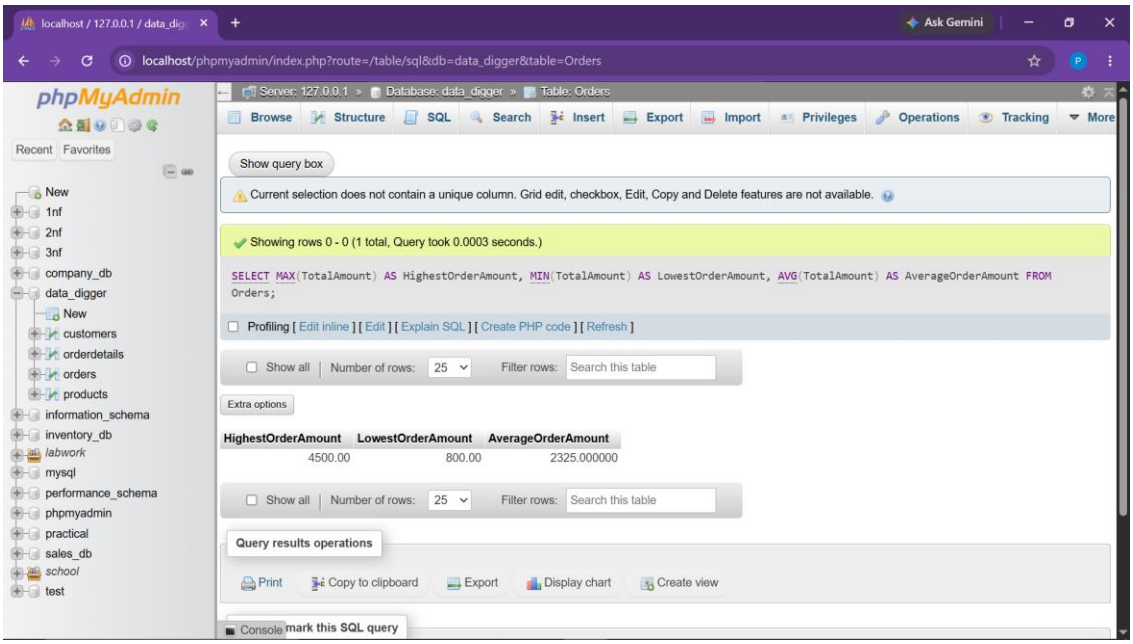
- **3.5 Retrieve Orders Placed in the Last 30 Days**

```
SELECT * FROM Orders
WHERE OrderDate >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```



- **3.6 Retrieve the Highest, Lowest, and Average Order Amount using Aggregate Functions**

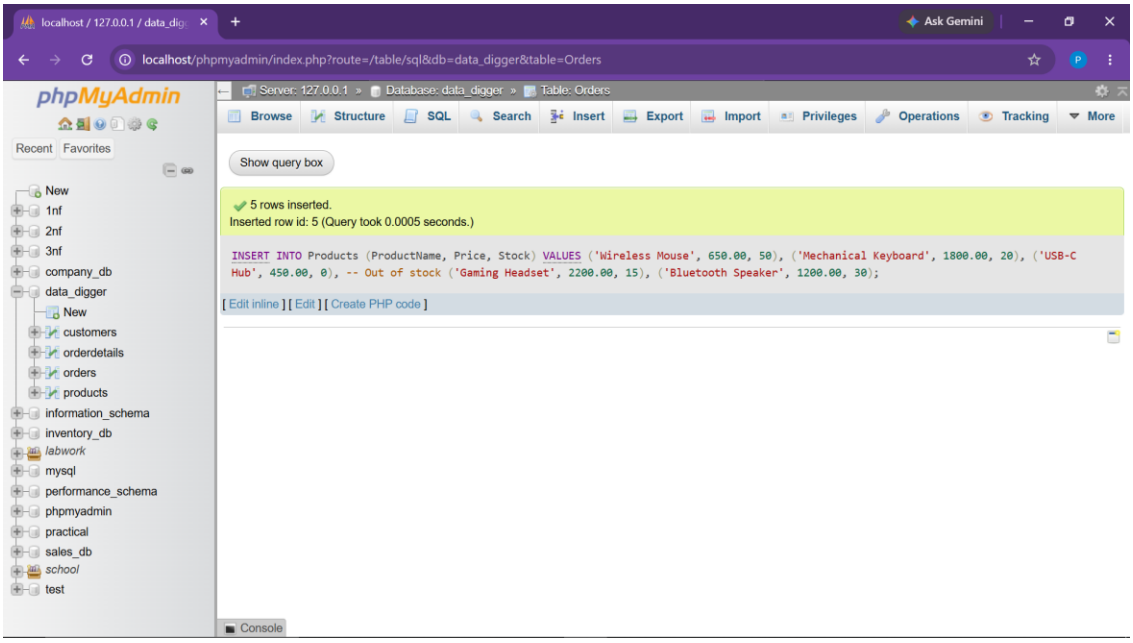
```
SELECT
    MAX(TotalAmount) AS HighestOrderAmount,
    MIN(TotalAmount) AS LowestOrderAmount,
    AVG(TotalAmount) AS AverageOrderAmount
FROM Orders;
```



4. PRODUCTS TABLE - DATA MANIPULATION & ANALYSIS

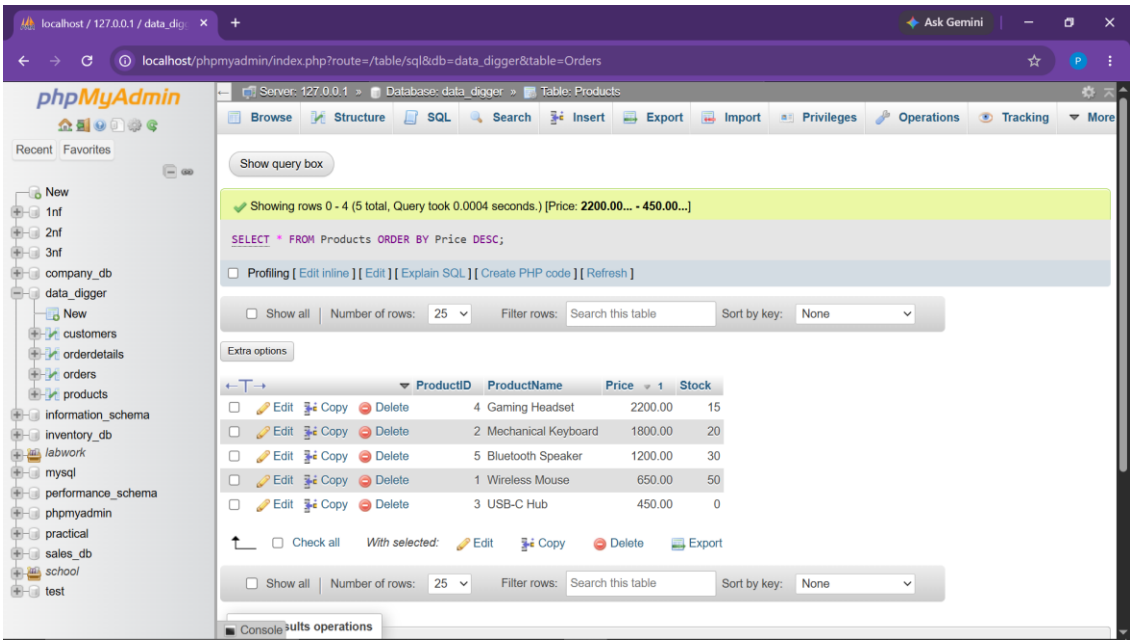
- **4.1 Insert at least 5 sample products into the Products table**

```
INSERT INTO Products (ProductName, Price, Stock) VALUES
('Wireless Mouse', 650.00, 50),
('Mechanical Keyboard', 1800.00, 20),
('USB-C Hub', 450.00, 0), -- Out of stock
('Gaming Headset', 2200.00, 15),
('Bluetooth Speaker', 1200.00, 30);
```



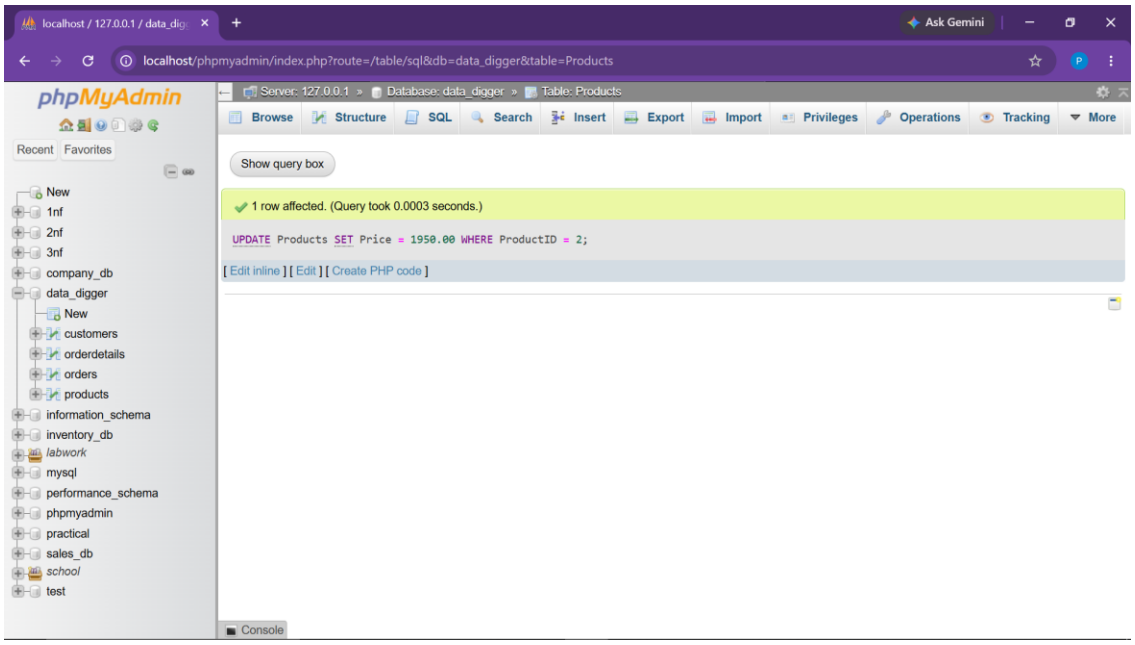
- **4.2 Retrieve all products sorted by price in descending order**

```
SELECT * FROM Products
ORDER BY Price DESC;
```



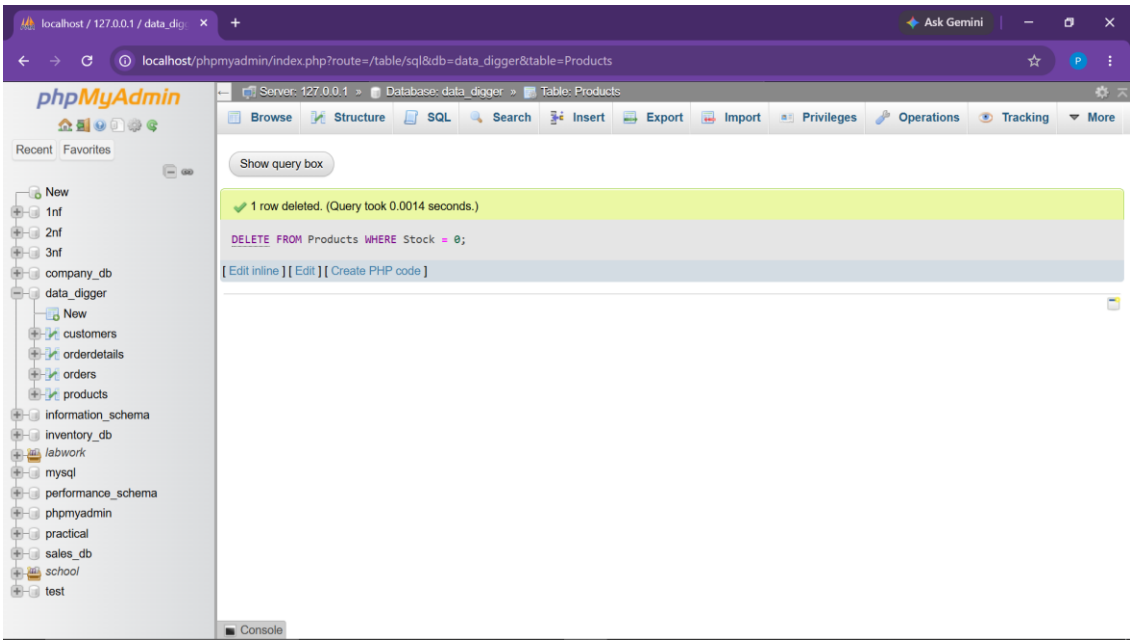
- **4.3 Update the price of a specific product**

```
UPDATE Products
SET Price = 1950.00
WHERE ProductID = 2;
```



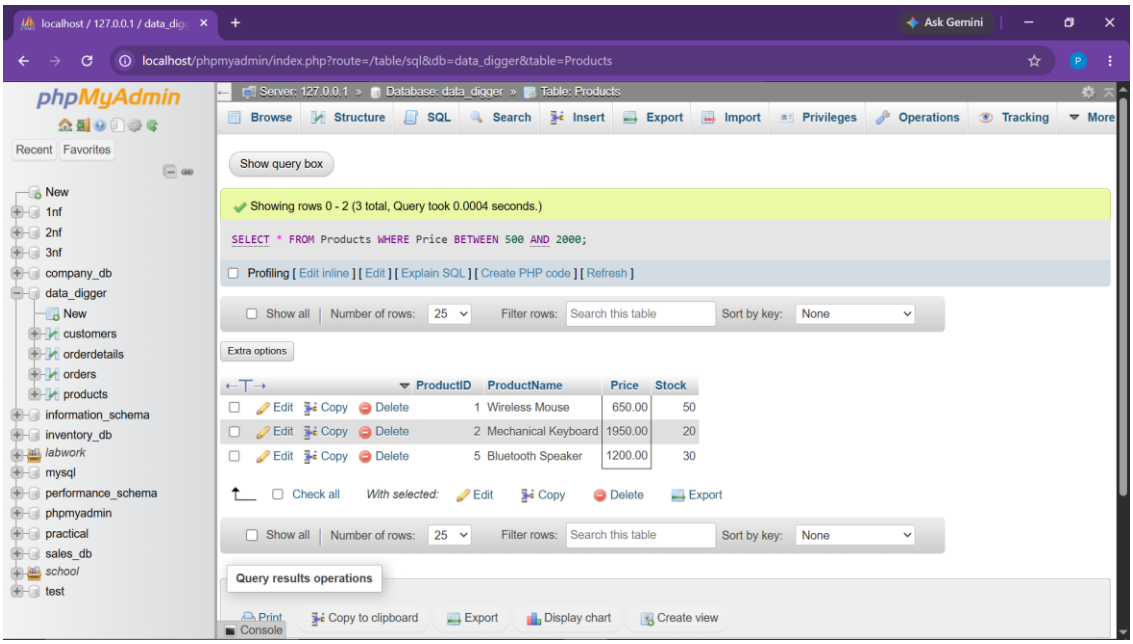
- **4.4 Delete a product if it's out of stock**

```
DELETE FROM Products
WHERE Stock = 0;
```



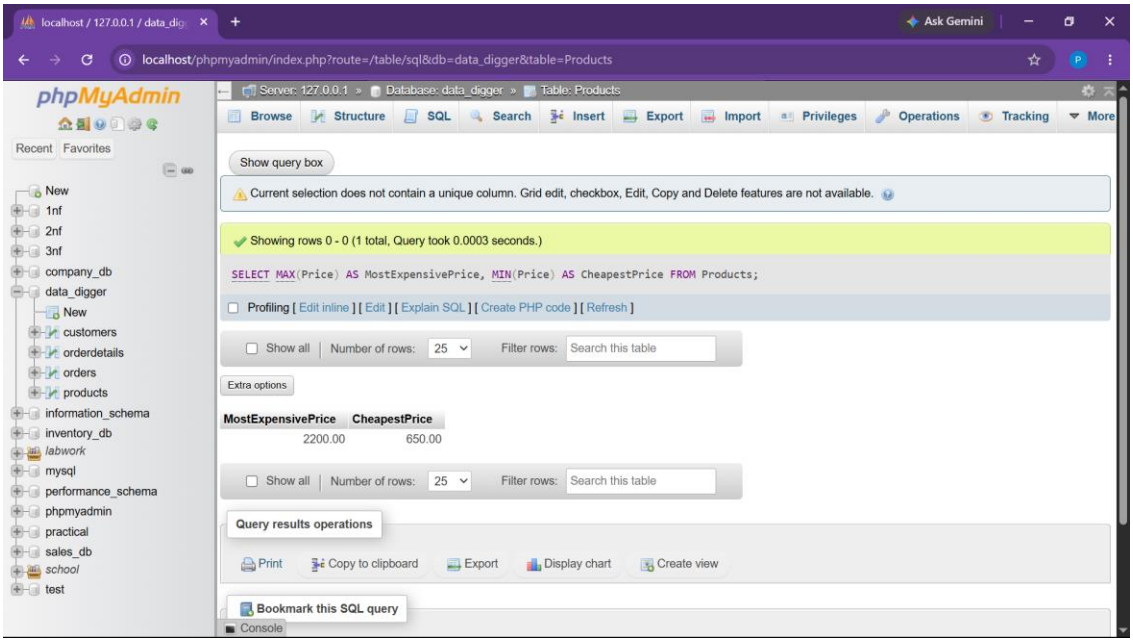
- **4.5 Retrieve products whose price is between ₹500 and ₹2000**

```
SELECT * FROM Products
WHERE Price BETWEEN 500 AND 2000;
```



- **4.6 Retrieve the most expensive and cheapest product using MAX() and MIN()**

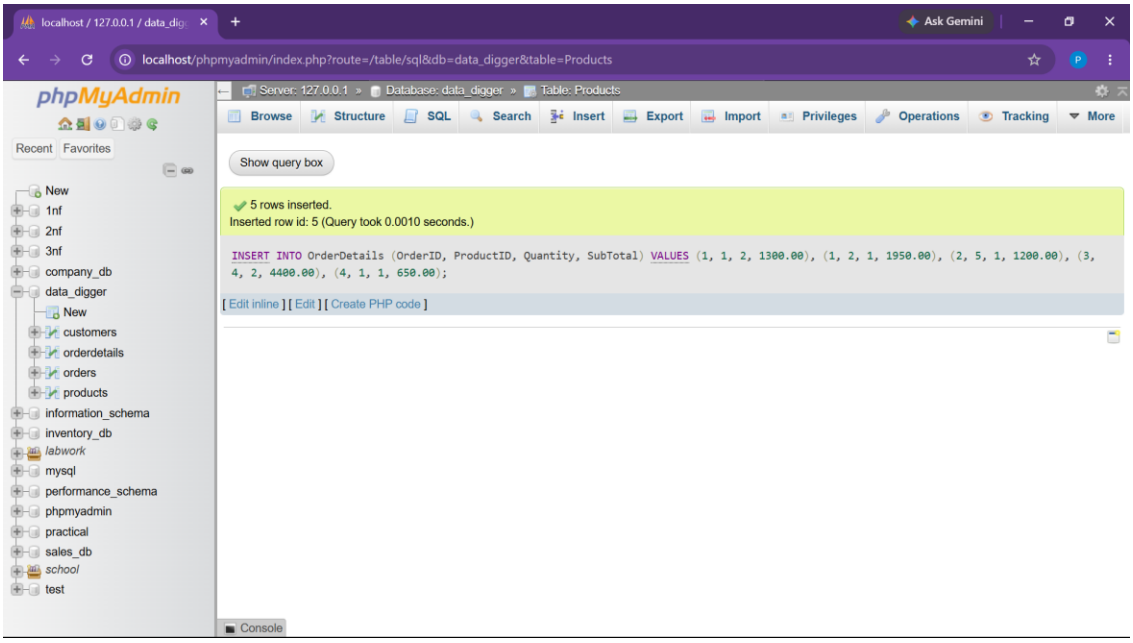
```
SELECT
    MAX(Price) AS MostExpensivePrice,
    MIN(Price) AS CheapestPrice
FROM Products;
```



5. ORDERDETAILS TABLE - RELATIONAL JOINS & AGGREGATIONS

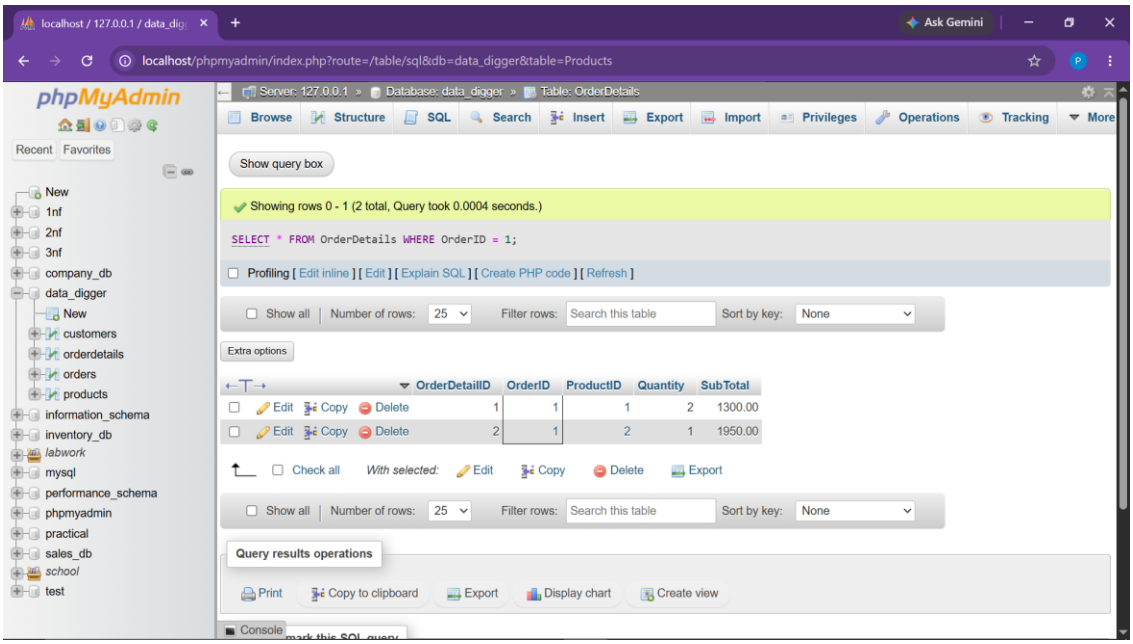
- **5.1 Insert Records into OrderDetails**

```
INSERT INTO OrderDetails (OrderID, ProductID, Quantity, SubTotal) VALUES
(1, 1, 2, 1300.00),
(1, 2, 1, 1950.00),
(2, 5, 1, 1200.00),
(3, 4, 2, 4400.00),
(4, 1, 1, 650.00);
```



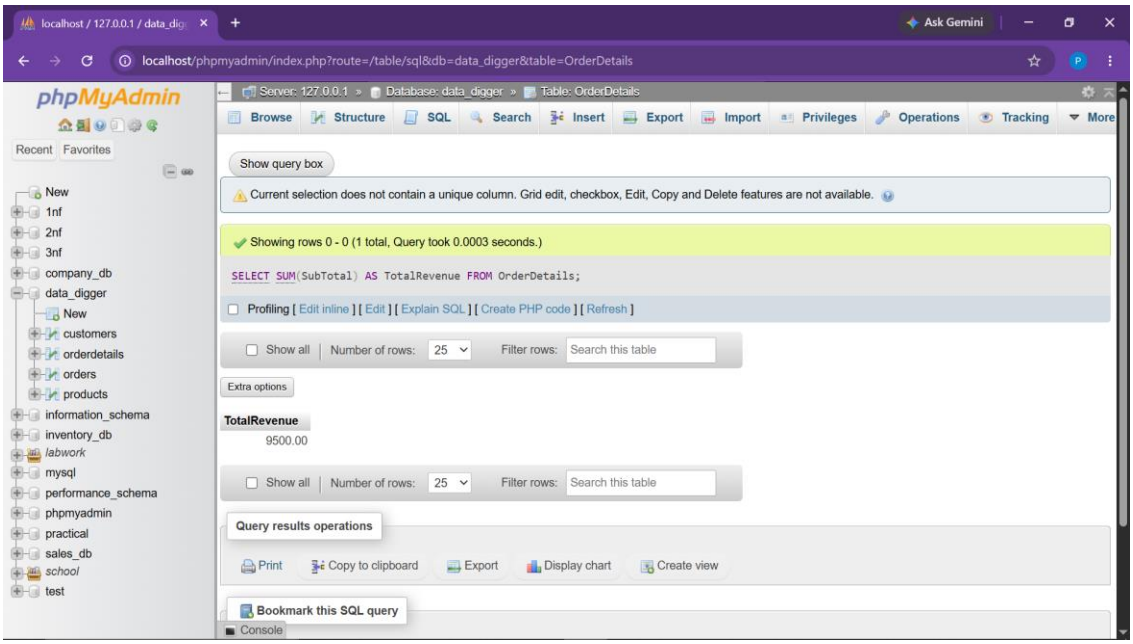
- **5.2 Retrieve Details for a Specific Order (OrderID = 1)**

```
SELECT * FROM OrderDetails
WHERE OrderID = 1;
```



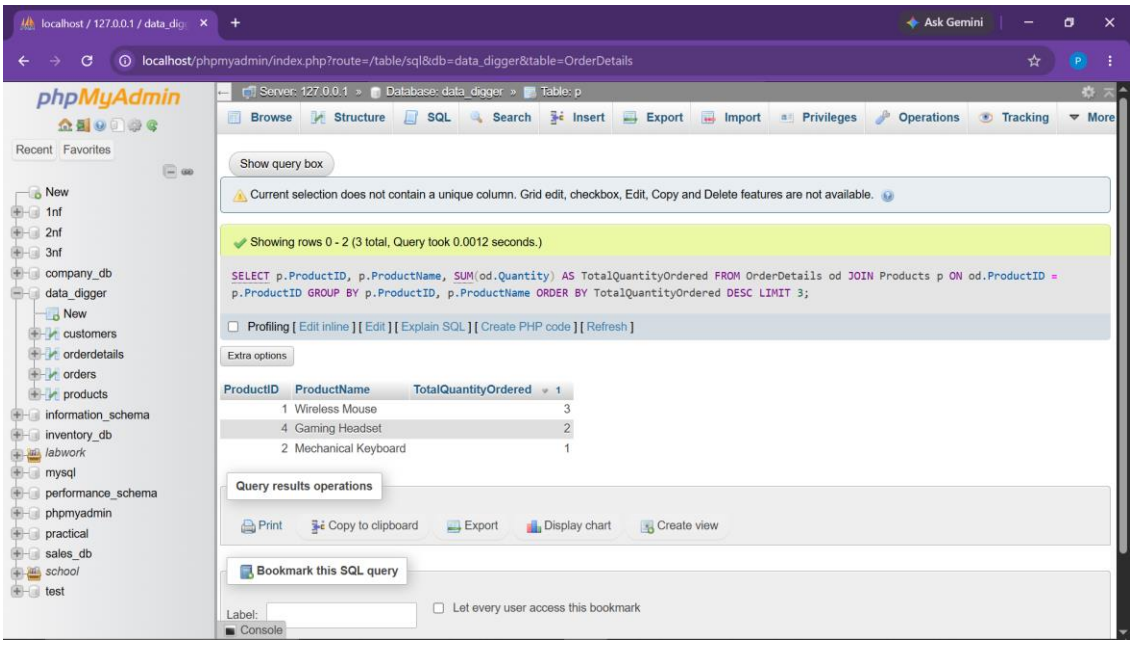
- ## 5.3 Calculate Total Revenue (SUM)

```
SELECT SUM(SubTotal) AS TotalRevenue
FROM OrderDetails;
```



- **5.4 Retrieve Top 3 Most Ordered Products (JOIN + GROUP BY + ORDER BY)**

```
SELECT
    p.ProductID,
    p.ProductName,
    SUM(od.Quantity) AS TotalQuantityOrdered
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY TotalQuantityOrdered DESC
LIMIT 3;
```



- **5.5 Count Number of Times a Specific Product Was Sold (ProductID = 1)**

```
SELECT COUNT(*) AS TimesSold
FROM OrderDetails
WHERE ProductID = 1;
```

